

Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd: Recuperação de Acurácia via Blocos *Bottleneck*

Rafael Silva de Souza

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

rafesilvadesouza@gmail.com

Mateus Grellert

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

Orientador

Abstract—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2 , 3×3), mas escalam mal para kernels grandes (5×5 , 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64 , 200 classes) sob um pipeline FP32 $\rightarrow$ *Quantization-Aware Training* (QAT) $\rightarrow$ INT8, comparando um baseline de kernel irrestrito, uma variante ingenuamente restrita a kernels 3×3 e uma variante que adiciona blocos *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$) a essa mesma restrição. A variante com *bottleneck* (AlexNetBottleneck) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (4,49 MB) — superando a restrição ingênua em mais de 4 pontos percentuais com $6\times$ menos parâmetros — e apresenta a menor queda de acurácia sob quantização INT8 de todo o projeto (-0,08 p.p.). Os resultados sugerem que compensação estrutural leve, e não kernels maiores, é a via mais eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd. Este é um relatório preliminar de iniciação científica voluntária, cobrindo um primeiro modelo representativo; comparações com as demais famílias de arquiteturas estudadas no projeto (compensação por *Fire*, *depthwise separable*, residual, entre outras) serão incluídas em versões futuras.

Index Terms—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, INT8

I. INTRODUÇÃO

O algoritmo de convolução de Winograd [2] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: a complexidade da transformada cresce de forma super-linear com o tamanho do filtro, tornando kernels grandes (5×5 , 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [1], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz drasticamente a capacidade do modelo,

já que cada camada passa a enxergar uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels no regime Winograd-amigável ($\leq 3\times 3$), em vez de simplesmente devolver kernels grandes ao modelo. Este relatório apresenta o primeiro resultado dessa investigação: blocos *bottleneck* no estilo ResNet [4] (redução de canais $1\times 1 \rightarrow$ convolução $3\times 3 \rightarrow$ expansão 1×1), aplicados a uma AlexNet com kernels restritos a 3×3 .

II. METODOLOGIA

A. Dados e configuração experimental

Todos os modelos foram treinados no Tiny ImageNet-200 (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico). O pipeline de treino segue três estágios: (i) treino FP32 do zero; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [6]; (iii) conversão para INT8 (*backend* fbgemm, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença entre ambos (*QAT drop*) como medida de robustez à quantização.

B. Arquiteturas comparadas

Três variantes, treinadas sob o mesmo protocolo, permitem isolar o efeito da restrição de kernel e o efeito da compensação:

- **AlexNet (irrestrita)** — arquitetura AlexNet padrão, kernels de até 11×11 , usada como referência interna do experimento.
- **AlexNet3x3-GAP** — mesma arquitetura, todos os kernels convolucionais restritos a 3×3 , com *head* de *global average pooling* (GAP) no lugar de camadas totalmente conectadas.
- **AlexNetBottleneck** — kernels igualmente restritos a 3×3 , mas cada estágio é substituído por um bloco *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$, razão de redução $4\times$) seguido de GAP.

O bloco *bottleneck* preserva o campo receptivo efetivo da convolução 3×3 central, mas usa as convoluções 1×1 para

comprimir e depois expandir os canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd.

III. RESULTADOS

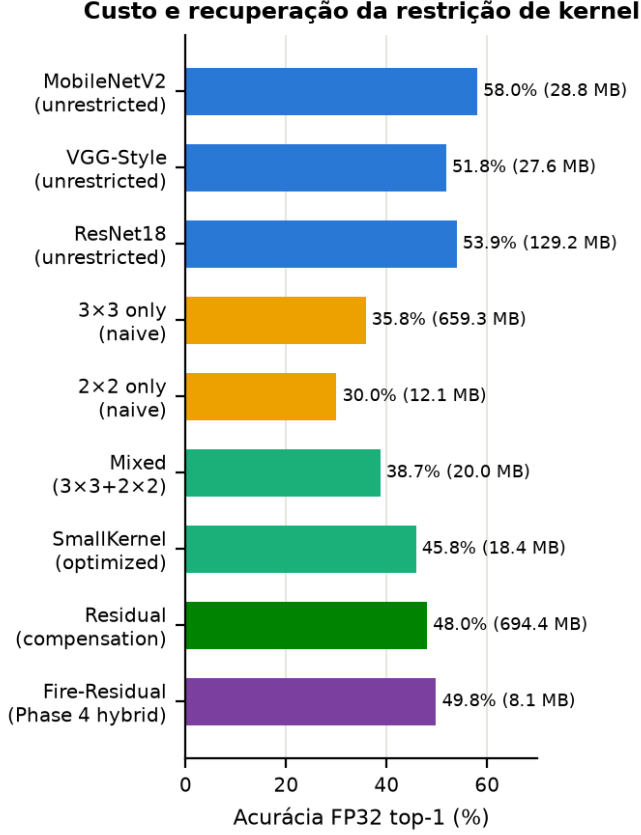


Fig. 1. Acurácia FP32 (top-1) por arquitetura, contrastando *baselines* de kernel irrestrito, restrição ingênua (3×3 , 2×2) e estratégias de compensação exploradas no projeto. SmallKernel, Residual e Fire-Residual pertencem a fases posteriores deste estudo e serão detalhadas em relatórios futuros.

A Tabela I resume os três modelos sob o mesmo protocolo de treino e avaliação.

TABLE I
COMPARAÇÃO ENTRE RESTRIÇÃO DE KERNEL INGÊNUA E COM COMPENSAÇÃO BOTTLENECK

Modelo	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNet (irrestrita)	57,82	32,20	26,28	-5,93
AlexNet3x3-GAP	2,30	40,32	37,02	-3,29
AlexNetBottleneck	0,385	44,62	44,54	-0,08

Dois resultados se destacam. Primeiro, a restrição de kernel por si só (AlexNet 3×3 -GAP) já supera o baseline irrestrito neste protocolo — o *head* GAP reduz drasticamente o *overfitting* da camada totalmente conectada original, com $25 \times$

menos parâmetros. Segundo, adicionar o bloco *bottleneck* sobre essa mesma restrição de kernel recupera mais 4,3 pontos percentuais de acurácia com $6 \times$ menos parâmetros que a variante GAP simples, e praticamente elimina a perda de acurácia sob quantização INT8.

A Fig. 1 situa esse resultado no panorama mais amplo do projeto, incluindo *baselines* não restritos (MobileNetV2, VGG-Style, ResNet18) e outras estratégias de recuperação de acurácia (*residual*, *Fire*) exploradas em fases posteriores.

A Fig. 2 mostra a queda de acurácia sob QAT→INT8 para arquiteturas representativas: modelos com compensação por *bottleneck* e *Fire* são substancialmente mais estáveis sob quantização do que restrições ingênuas de kernel.

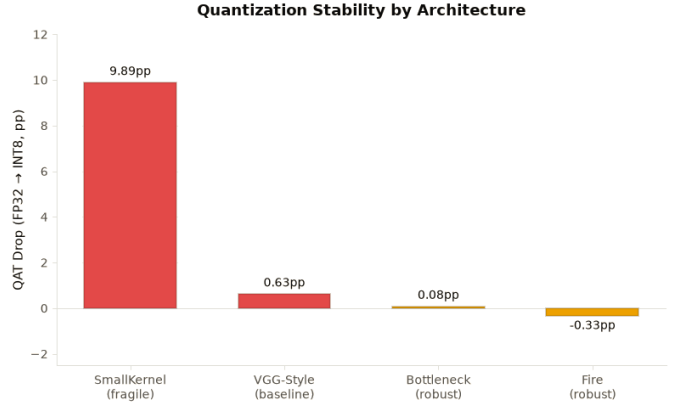


Fig. 2. Queda de acurácia (p.p.) de FP32 para INT8 por arquitetura. Mecanismos de compensação (*bottleneck*, *Fire*) mantêm a acurácia quase intacta sob quantização, ao contrário de restrições de kernel sem compensação.

Por fim, a Fig. 3 agrega, em todo o conjunto de arquiteturas do projeto, a acurácia por megabyte de modelos Winograd-amigáveis (kernel $\leq 3 \times 3$, com compensação) contra arquiteturas que dependem de kernels maiores.

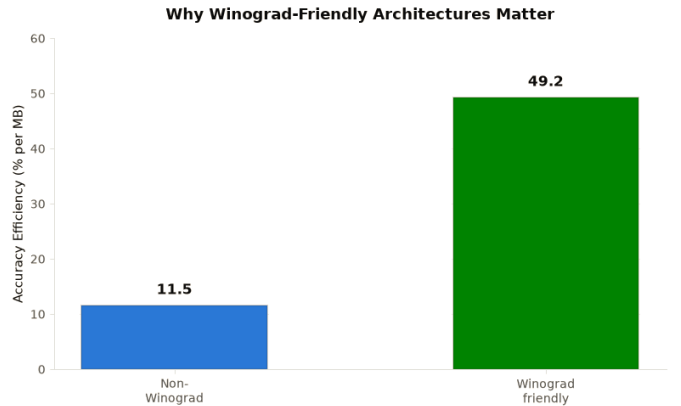


Fig. 3. Eficiência de acurácia (% por MB), agregada por família de arquitetura Winograd-amigável vs. não amigável, em todo o conjunto de modelos avaliados no projeto.

IV. DISCUSSÃO

O padrão observado — restrição de kernel isolada perde acurácia, mas compensação estrutural leve a recupera quase

por completo — é consistente com a literatura de arquiteturas eficientes: blocos *bottleneck* [4] e módulos *Fire* [3] foram originalmente propostos para reduzir parâmetros sem restrição de kernel, mas o presente resultado indica que o mesmo mecanismo é igualmente eficaz quando o kernel já está restrito por uma necessidade externa (compatibilidade com Winograd), e não apenas por economia de parâmetros.

A robustez do AlexNetBottleneck sob quantização (-0,08 p.p.) também é notável frente à variante GAP simples (-3,29 p.p.): a hipótese de trabalho é que as convoluções 1×1 do *bottleneck* produzem ativações com distribuições mais regulares, facilitando a calibração dos parâmetros de quantização — hipótese que pretendemos testar diretamente em trabalhos futuros deste projeto.

Uma primeira medição de hardware (Fase 6 do projeto, em andamento) já fornece um dado direto de latência para o AlexNetBottleneck: 0,81 ms em FP32 e 1,19 ms em INT8 (*batch*=1, GPU NVIDIA RTX 4060 Laptop), obtidos pelo mesmo *profiling* em nível de camada aplicado às convoluções 3×3 isoladas do bloco *bottleneck*. O rastreamento do kernel CUDA efetivamente selecionado pela cuDNN confirma que essas convoluções são executadas por um kernel Winograd $F(2 \times 2, 3 \times 3)$ real — validando que a arquitetura é de fato utilizável pela aceleração que motiva a restrição de kernel —, mas apenas em *batches* pequenos (1 e 8); em *batch*=64, a mesma convolução 3×3 passa a ser executada por um kernel TF32 sobre *tensor cores*, e a latência medida nesse regime deixa de mostrar qualquer vantagem de $k = 3$ sobre kernels maiores. Isso sugere que, em GPUs equipadas com *tensor cores*, a aceleração Winograd compete diretamente com a aceleração de *tensor cores* e tende a ser preterida em *batches* maiores — uma limitação da GPU como plataforma de validação, não do princípio de restrição de kernel em si: o acelerador FPGA que motiva este projeto não possui *tensor cores*, e a redução no número de multiplicações que o Winograd oferece se converteria em economia de área e potência independentemente do tamanho do *batch*.

V. CONCLUSÃO E TRABALHOS FUTUROS

Restringir o kernel de uma AlexNet a 3×3 é custoso para a acurácia, mas esse custo é, em grande parte, recuperável por um mecanismo de compensação leve — o bloco *bottleneck* — que não reintroduz nenhum kernel maior que 3×3 e, portanto, preserva integralmente a compatibilidade com aceleração Winograd. Com apenas 0,385M parâmetros, o AlexNetBottleneck atinge 44,62% de acurácia top-1 em FP32 e mantém 44,54% após quantização INT8, tornando-se o ponto mais eficiente do estudo até aqui em acurácia por megabyte entre modelos estáveis sob quantização.

Este relatório cobre apenas o primeiro modelo de uma investigação mais ampla, que inclui outras famílias de compensação (*Fire*, *depthwise separable*, *residual*, *squeeze-and-excitation*), arquiteturas híbridas finais combinando múltiplos mecanismos, e uma extensão a detecção de objetos (SSD sobre PASCAL VOC). Versões futuras deste relatório incorporarão essas comparações. Uma primeira medição direta

de latência em hardware real para o AlexNetBottleneck já está disponível (ver Discussão) e será estendida às demais famílias de arquitetura e a medições de consumo de energia em trabalhos futuros.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [2] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” *arXiv preprint arXiv:1602.07360*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [6] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] Y. Le and X. Yang, “Tiny ImageNet visual recognition challenge,” Stanford CS231N Report, 2015.